

Atenção:

- Está sendo fornecido um arquivo Python com nome `paa2025_prova.py` junto com esse enunciado.
- As soluções das questões 1 e 4 deverão ser escritas no espaço destinado para elas nos comentários das respectivas funções do arquivo Python.
- As soluções das questões 2 e 3 deverão ser escritas nas suas respectivas funções do arquivo Python.
- O nome do arquivo Python e a assinatura das funções não devem ser alteradas.
- Caso seja necessário definir uma classe ou função auxiliar, escreva no escopo da função da questão que você estiver resolvendo.

Questão 1 (2 pontos)

Determine uma função $f(n)$ para o código a seguir tal que $T(n) = \theta(f(n))$. Explique como a solução foi encontrada.

```
int compute(int v[], int left, int right) {
    int n = right - left + 1;
    if (n <= 1) return 0;

    int mid = (left + right) / 2;

    int sum = 0;
    sum += compute(v, left, mid);
    sum += compute(v, mid + 1, right);
    sum += compute(v, left + n/4, mid + n/4);
    sum += compute(v, mid - n/4, right - n/4);

    int temp[n];
    int tempSize = 0;
    for (int i = left; i <= right; i++) {
        int pos = 0;
        while (pos < tempSize && temp[pos] < v[i]) {
            pos++;
        }

        for (int j = tempSize; j > pos; j--) {
            temp[j] = temp[j-1];
        }
    }
}
```

```
        temp[pos] = v[i];
        tempSize++;
        sum++;
    }

    for (int i = 0; i < n; i++) {
        v[left + i] = temp[i];
    }

    return sum;
}
```

Questão 2 (2 pontos)

Uma empresa de logística mantém produtos em um armazém circular organizados em ordem crescente. Cada produto é identificado por um código numérico. Quando o armazém sofre rotação para otimização de espaço, a sequência ordenada é "rotacionada" - alguns elementos do final vão para o início, mantendo a ordem relativa. Por exemplo, a sequência [10, 20, 30, 40, 50] após rotação de 2 posições se torna [40, 50, 10, 20, 30]. Desenvolva um algoritmo eficiente para encontrar um produto específico neste armazém rotacionado, retornando a sua posição ou -1 caso não exista. O algoritmo deverá ter complexidade $O(\log n)$.

Exemplos:

Entrada	Saída	
<i>products</i>	<i>k</i>	
[40, 50, 60, 10, 20, 30]	10	3
[4, 5, 6, 7, 0, 1, 2]	0	4
[30, 40, 50, 10, 20]	35	-1

Questão 3 (3 pontos)

Um serviço de streaming precisa manter atualizado em tempo real um ranking com as k músicas mais tocadas. A cada nova reprodução de uma música, um evento é gerado para atualizar o ranking. Como o volume de eventos é imenso, manter todas as músicas disponíveis ordenadas é inviável. Projete um algoritmo que processe um stream com n eventos, contendo em cada um o identificador numérico da música, e produza o ranking de k posições em $O(n \log k)$, onde $k \ll n$.

Exemplos:

Entrada <i>events</i>	Saída	
	<i>k</i>	
[5, 1, 2, 1, 3, 2, 1, 4, 2, 1]	3	[(1, 4), (2, 3), (3, 1)]
[30, 10, 10, 20, 20, 10]	2	[(10, 3), (20, 2)]
[2, 5, 5, 5, 5, 1, 3]	4	[(5, 4), (1, 1), (2, 1), (3, 1)]
[100, 200, 100, 300, 200, 100, 200]	1	[(100, 3)]

Questão 4 (3 pontos)

Considerando a tabela hash com endereçamento aberto, responda as perguntas a seguir:

- Explique como é a sua estrutura de dados, e como funcionam os algoritmos de busca, inserção e remoção.
- Compare as estratégias de hash duplo e sondagem quadrática.
- Explique o que é o processo de resize e re-hashing, e avalie a sua complexidade.